

DrawingsDreamer: A Unified Multi-View Engineering Drawings Generation Model

Shurui Liu¹ Weide Chen² Changwang Yi¹ Ancong Wu^{1*}

¹ School of Computer Science and Engineering, Sun Yat-sen University, China

² School of Intelligent Systems Engineering, Shenzhen Campus of Sun Yat-sen University, China

{liushr29, chenwd56, yichw}@mail2.sysu.edu.cn

wuanc@mail.sysu.edu.cn

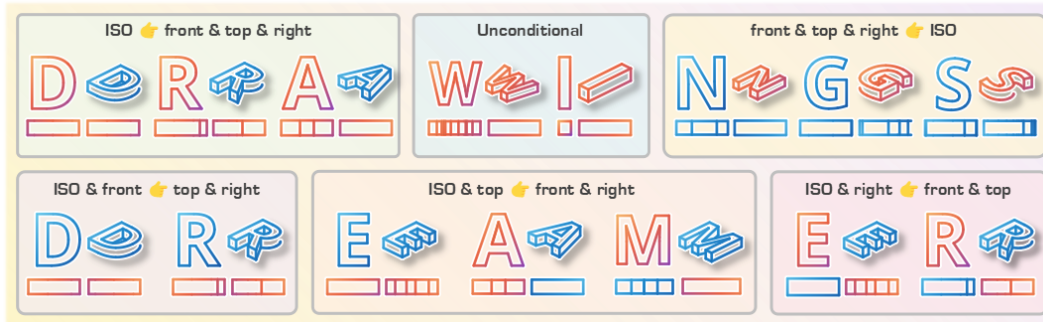


Figure 1: **DrawingsDreamer** handles a full spectrum of multi-view modeling tasks. **The orange gradients denote the outputs, while the blue gradients represent the inputs.** In each example, the multi-view layout arranges the Top, Front, and Right orthographic views at the top-left, bottom-left, and bottom-right, respectively, with the Isometric view situated at the top-right.

Abstract

Scalable Vector Graphics (SVG) are essential for modern industrial Computer-Aided Design (CAD). However, existing autoregressive SVG generation models are predominantly tailored for artistic creation and struggle to maintain the rigorous geometric fidelity and cross-view spatial alignment required for engineering drawings. To bridge this gap, we introduce **DrawingsDreamer**, a unified Large Language Model (LLM)-driven framework for multi-view vector-based engineering drawings generation. By formulating the generation of multi-view engineering drawings purely as a sequence modeling task, we eliminate the need of raster image encoders. We propose a Streamlined Representation utilizing hierarchical postfix tokenization, which guides the model to establish local geometric coordinates before assigning semantic boundaries. Optimized via a progressive task-aware curriculum schedule, **DrawingsDreamer** effectively transitions from localized structural repair to macroscopic generation in a unified model. Extensive experiments demonstrate that our unified model achieves strong performance in both geometric fidelity and syntactic accuracy across diverse conditional and unconditional generation tasks.

1 Introduction

Scalable Vector Graphics (SVG) have long served as the *de facto* standard in modern digital design, spanning applications from user interfaces (UI) to complex industrial Computer-Aided Design

* Corresponding author.

(CAD) systems. Their widespread adoption is driven by resolution independence, compact file sizes, and the capacity for precise, parametric control over geometric primitives such as polygons and Bézier curves. While recent advancements in generative AI have revolutionized visual synthesis, the creation of highly structured, multi-view engineering drawings remains a formidable challenge. As the foundation of modern manufacturing, engineering drawings demand strict structural validity, exact geometric parameterization, and rigorous cross-view spatial alignment. These qualities differ fundamentally from the requirements of general artistic creation.

Existing approaches to SVG generation largely fall short of these industrial demands. Optimization-based methods [36, 29] often struggle with prohibitive computational overhead and tend to produce unstructured, dense anchor points that destroy the editability inherent to the SVG format. Conversely, autoregressive methods and LLM-driven pipelines [48, 1, 62] have demonstrated remarkable scalability by treating SVG generation as a sequence modeling task. However, bottlenecked by limited context lengths and training corpora overwhelmingly skewed towards artistic domains (e.g., icons, fonts, and illustrations) [59, 41], these models fail to capture the long-range coordinate dependencies and spatial constraints required for engineering CAD drafts. Concurrently, the broader CAD generation community has predominantly focused on synthesizing 3D Boundary Representations (BRep) or meshes from point clouds and images [57, 61], leaving the direct generation of the 2D drawings themselves entirely unexplored.

Aligning with the broader consensus that scalable, general-purpose sequence modeling ultimately outpaces hand-crafted optimization heuristics, we frame the synthesis of 3D engineering drawings purely as a 2D sequence generation paradigm. In this paper, we propose **DrawingsDreamer**, the first unified multi-view engineering drawings generation model. By projecting 3D CAD objects into a canonical set of principal views, we deliberately avoid harnessing any raster image encoders. Instead, we rely entirely on the robust spatial reasoning capabilities of LLMs operating over discrete coordinate tokens. To effectively map continuous geometric spaces to discrete vocabularies, we introduce a *Streamlined Representation* featuring a novel hierarchical postfix tokenization strategy. This formulation compels the model to robustly establish continuous local coordinates prior to encapsulating them with semantic boundaries, thereby stabilizing the generation of complex spatial structures.

To facilitate the training of our model, we systematically construct a multi-view aligned engineering drawings corpus by rendering standard projections from existing 3D CAD collections. Leveraging this structured data, we optimize **DrawingsDreamer** through a progressive, task-aware curriculum schedule, empowering a single unified model to seamlessly transition between microscopic topological repair, partial view completion, and unconstrained macroscopic drawings imagination.

In summary, our core contributions are as follows:

- We propose a *Streamlined Representation* utilizing a hierarchical postfix tokenization strategy. This formulation effectively maps continuous geometric spaces into a discrete, LLM-friendly vocabulary, fundamentally stabilizing the autoregressive generation of complex spatial coordinates.
- We introduce **DrawingsDreamer**, the first unified framework capable of generating highly structured, multi-view parametric engineering drawings. By optimizing the model via a progressive, task-aware curriculum schedule, we empower a single framework to seamlessly transition between microscopic topological repair and macroscopic spatial generation.
- Extensive evaluations demonstrate that **DrawingsDreamer** achieves state-of-the-art performance in both geometric fidelity and syntactic accuracy across diverse conditional and unconditional multi-view generation tasks.

2 Related Work

2.1 SVG Generation

The automated generation of vector graphics has witnessed a significant paradigm shift from instance-level optimization to large-scale sequence modeling. Early optimization methods [36, 29] iteratively refine SVG parameters by minimizing photometric loss via differentiable rasterizers. However, these approaches rely heavily on iterative optimization for individual instances. Consequently, they suffer

from prohibitive computational overhead and often yield unstructured outputs lacking geometric fidelity. In contrast, driven by the shift towards large-scale data and general-purpose computation, learning-based SVG generation has witnessed significant progress. Advancements range from the curation of extensive vector datasets [54, 7, 20, 63] to the development of generative pipelines built upon RNNs [44, 13, 40, 16] and VAEs [5, 34, 46, 45, 47]. More recently, the paradigm has shifted towards treating SVG generation as a sequence modeling task by leveraging Large Language Models (LLMs) [48, 1, 62, 8, 49], which demonstrate exceptional generative flexibility across diverse domains [25, 67, 43, 66, 17, 12, 9].

Despite these advances, existing LLM-driven SVG generation models [59, 41, 56, 63] remain bottlenecked by limited context lengths and training corpora restricted to artistic or visual creation. Consequently, generating highly structured engineering drawings with aligned multiview representations remains an unexplored frontier, limiting the adoption in professional CAD workflows. To bridge this gap, we propose the first unified framework for generating consistent multiview parametric engineering drawings, effectively scaling these capabilities to complex industrial applications.

2.2 Computer-Aided Design

The domain of Computer-Aided Design (CAD) has rapidly evolved from foundational datasets [57, 21, 55] to sophisticated generative pipelines [28, 60, 26, 23, 27, 58, 2, 57, 61, 24, 32, 18, 42, 6, 30, 51, 2, 64, 19]. Current methods predominantly focus on reverse engineering, generating Boundary Representation (BRep) models from modalities like point clouds [57, 61, 24, 32, 18, 42], images [6, 30], or text [51, 2, 64, 19].

Furthermore, while some techniques reconstruct CAD models from static engineering drawings [11, 22, 33, 52, 10, 4, 14, 38, 53, 65, 39], generating the drawings themselves remains entirely unexplored. As the fundamental blueprints of modern manufacturing, automating the creation of these drawings represents a critical missing link. To address this void and facilitate the training of generative models, we establish a scalable pipeline to derive structured, multi-view aligned 2D drawings directly from standard 3D CAD databases.

3 Method

In this section, we present the architecture and training pipeline of **DrawingsDreamer**. We first introduce our *Streamlined Representation* (Section 3.1), which employs a hierarchical postfix tokenization scheme to effectively map continuous parametric geometries into a discrete, LLM-friendly vocabulary. Building upon this formulation, we then detail our unified instruction-tuning paradigm and progressive task-aware curriculum schedule (Section 3.2). This dynamic optimization strategy empowers a single foundation model to seamlessly master both microscopic topological validity and rigorous cross-view spatial alignment.

3.1 Streamlined Representation for Multi-View Engineering Drawings

To effectively harness the scaling capabilities of LLMs for engineering drawings generation, we formulate the synthesis of engineering drawings as a pure sequence modeling paradigm. Specifically, a 3D CAD object $\mathcal{M}$ is projected into a canonical set of four principal views $\mathcal{V} = \{v_{\text{front}}, v_{\text{right}}, v_{\text{top}}, v_{\text{iso}}\}$. This projection provides a complete, unambiguous geometric description while deliberately avoiding the computational overhead associated with rendering and processing rasterized images via visual encoders. We encode this geometry into a *Streamlined Representation*, a compact training format optimized purely for LLM comprehension.

Autoregressive Postfix Tokenization. While prior works typically adopt a prefix-based tokenization scheme [63], we introduce a hierarchical *postfix tokenization* strategy. The model is compelled to first regress the geometric coordinates, subsequently encapsulating them with a semantic boundary token (e.g., `<end_curve>`). To maintain a streamlined vocabulary and reduce the model token search space, the standard SVG close path command (Z or z) is not assigned a unique token. Instead, our parser explicitly converts it into a line sequence connecting the current position back to the subpath starting coordinate, terminated by `<end_line>`. The geometric definitions of our structural vocabulary are summarized in Table 1.

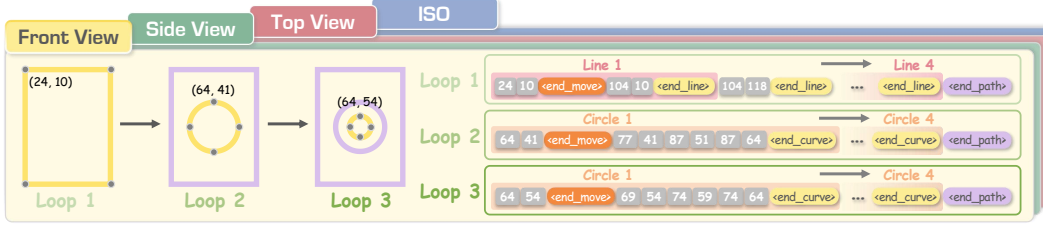


Figure 2: Visualization of the Streamlined Representation, detailing its hierarchical data organization. Continuous coordinates are grouped into primitive boundaries (e.g., lines and curves), formed into closed loops, and ultimately structured into unified view-level sequences.

Table 1: Vocabulary of structural boundary tokens and their geometric semantics.

Hierarchy Level	Token Format	Geometric Semantics
Primitive Boundaries	x, y <code><end_move></code>	Move (M): Initializes a new subpath at coordinate (x, y) .
	x, y <code><end_line></code>	Line (L): Draws a straight segment from the current position to (x, y) .
	x_1, y_1, x_2, y_2, x, y <code><end_curve></code>	Cubic Bézier (C): Draws a curve ending at (x, y) using control points (x_1, y_1) and (x_2, y_2) .
Macro Boundaries	<code><end_path></code>	Path Boundary: Terminates a continuous geometric path $\mathcal{P}$.
	<code><end_view></code>	View Boundary: Terminates a spatial projection view sequence S_v .
	<code><mask_path></code>	Mask Placeholder: Represents a masked path $\mathcal{P}_{\text{mask}} \in S_{\text{context}}$ during training.

Coordinate Normalization. To bridge continuous geometric spaces with discrete token vocabularies, coordinates are globally scaled to preserve cross-view spatial alignment (with the isometric view scaled independently) and quantized into N discrete integer bins. The mathematical formulation is deferred to Appendix.

Unified Multitask Sequence Construction. As visualized in Figure 2, rather than statically concatenating all views, we serialize the quantized paths into a flexible layered sequence designed to support a unified multitask training framework. The sequence is built hierarchically: coordinates are grouped by primitive boundary tokens (e.g., `<end_curve>`), primitives are grouped into paths terminated by `<end_path>`, and paths are aggregated into a complete view sequence S_v , strictly terminated by its corresponding view boundary token (e.g., `<end_front>`).

To empower the LLM with diverse generative capabilities, we dynamically construct the final drawings sequence as an instruction following prompt:

$$S_{\mathcal{M}} = [\mathcal{I}, S_{\text{context}}, S_{\text{target}}]. \quad (1)$$

Here, $\mathcal{I}$ acts as a task specific instruction. During training, we employ a curriculum learning strategy, dynamically sampling from a rich set of macroscopic generation and microscopic restoration tasks. To formalize these tasks, we define the orthographic view set as $\mathcal{V}_{\text{ortho}} = \{v_{\text{front}}, v_{\text{right}}, v_{\text{top}}\}$, and the complete view set as $\mathcal{V} = \mathcal{V}_{\text{ortho}} \cup \{v_{\text{iso}}\}$. The configurations of S_{context} and S_{target} are systematically varied across epochs as detailed in Table 2.

Table 2: Summary of unified multitask configurations. $\mathcal{V}_{\text{ortho}}$ denotes the set of orthographic views, and v_i represents a single orthographic view.

Task Configuration	Input Context (S_{context})	Target Response (S_{target})
Unconditional Generation	$\emptyset$	$\mathcal{V}$
Isometric to Orthographic	$\{v_{\text{iso}}\}$	$\mathcal{V}_{\text{ortho}}$
Orthographic to Isometric	$\mathcal{V}_{\text{ortho}}$	$\{v_{\text{iso}}\}$
Partial View Completion	$\{v_{\text{iso}}, v_i\} \mid v_i \in \mathcal{V}_{\text{ortho}}$	$\mathcal{V}_{\text{ortho}} \setminus \{v_i\}$
Path Level Semantic Masking	$S_{\text{available}} \setminus \{\mathcal{P}_{\text{mask}}\}$	$\{\mathcal{P}_{\text{mask}}\}$

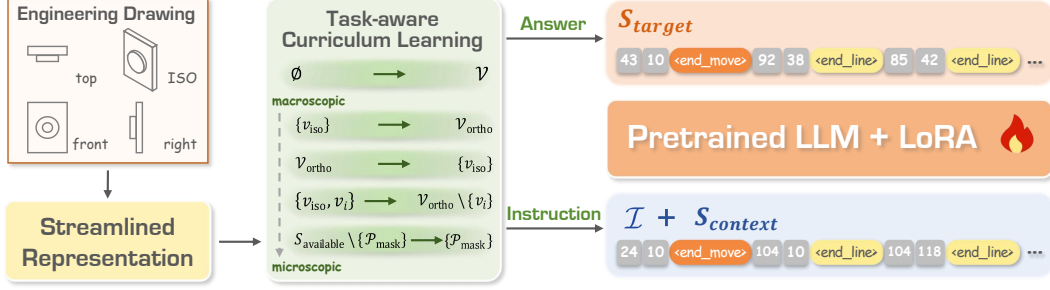


Figure 3: **Overview of the proposed training framework.** We cast diverse multi-view generation sub-tasks into a unified sequence modeling paradigm, guided by a progressive microscopic-to-macroscopic optimization schedule. The task sampling distribution dynamically transitions from microscopic local topology learning (Stage I) to partial view alignment (Stage II), and ultimately to macroscopic global spatial translation (Stage III) across training epochs.

This unified formulation ensures that a single generative model can seamlessly transition between full spatial projection and localized topological repair. Crucially, as outlined in Table 2, this structure naturally accommodates our local geometric restoration task.

3.2 Instruction-Tuning with Task-Aware Curriculum Schedule

To empower the LLM with comprehensive multi-view spatial reasoning, we cast all generation sub-tasks detailed in Table 2 into a unified prediction framework, as illustrated in Figure 3. Rather than training specialized models, we employ a single pre-trained Llama-family [48] causal language model. To achieve this efficiently, we apply Low-Rank Adaptation (LoRA) [15] alongside standard instruction-tuning practices. Specifically, we compute the cross-entropy loss $\mathcal{L}$ exclusively over the generative target S_{target} :

$$\mathcal{L} = - \sum_{t=1}^{|S_{\text{target}}|} \log P(x_t | x_{<t}, \mathcal{I}, S_{\text{context}}; \Theta), \quad (2)$$

where Θ denotes the trainable LoRA parameters. By masking the instruction $\mathcal{I}$ and context S_{context} during optimization, this unified objective allows the model to seamlessly alternate between diverse generation modes within the same training batch without gradient interference.

Microscopic-to-Macroscopic Task-Aware Curriculum. Empirical evidence suggests that directly optimizing LLMs on complex, weakly-conditioned spatial generation tasks often leads to sub-optimal convergence and hallucinated topologies [3, 50]. To address this, we introduce a progressive, task-aware curriculum schedule that dynamically adjusts the sampling distribution of tasks across training epochs.

Crucially, we treat *Unconditional Generation* ($\emptyset \rightarrow \mathcal{V}$) as a continuous base task. Its sampling weight remains continuously active throughout the entire training process to robustly anchor the model’s fundamental data distribution and generative capacity. Concurrently, the distribution of the remaining conditional tasks follows a smoothed optimization trajectory spanning three progressive stages:

- **Stage I: Local Syntax and Topology (Early Epochs).** The conditional sampling distribution heavily favors microscopic structural tasks, specifically *Path Level Semantic Masking*. This initial phase compels the model to establish robust coordinate syntax and localized geometric priors before tackling complex global spatial reasoning.
- **Stage II: Partial View Alignment (Middle Epochs).** As local geometric stability is acquired, we dynamically shift the conditional weights towards *Partial View Completion* (e.g., inferring two missing orthographic views given the isometric and one available orthographic view). This stage forces the model to learn cross-view spatial alignment and correlation anchored by strong contextual cues.

- **Stage III: Global Spatial Translation (Late Epochs).** In the final stage, the distribution transitions predominantly to macroscopic spatial tasks, specifically *Iso-to-Ortho* and *Ortho-to-Iso* translations. Having progressively mastered local topology and partial alignment, the model is now optimized for rigorous 3D structural reasoning and deterministic projective geometry inference from highly compressed contexts.

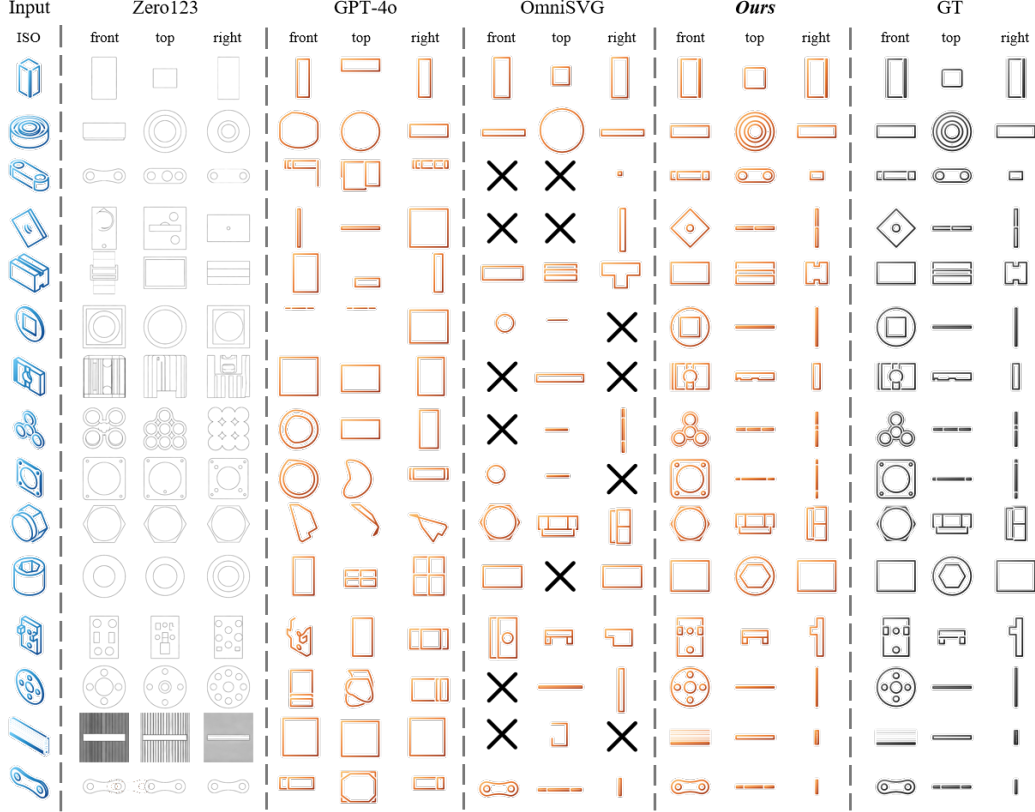


Figure 4: Qualitative results on Isometric to Orthographic translation ($\{v_{\text{iso}}\} \rightarrow \mathcal{V}_{\text{ortho}}$).

By adopting this microscopic-to-macroscopic optimization trajectory, our unified model sequentially builds its geometric reasoning capacity, seamlessly scaling from localized structural repair to unconstrained multi-view spatial imagination.

4 Experiments

4.1 Experimental Setup

Implementation Details. We instantiate DrawingsDreamer using LLaMA-3.2-3B [37] as the base LLM, chosen for its strong sequence modeling capabilities and competitive performance among open-source foundation models. To ensure parameter-efficient fine-tuning, we employ LoRA [15] with a rank of $r = 8$ and $\alpha = 32$. For the hardware setup, models with parameters up to 3B are trained on a cluster of eight NVIDIA RTX 4090 GPUs, while larger models are trained on eight NVIDIA L40 GPUs. We employ the AdamW optimizer [35] with a batch size of 16. We apply a cosine annealing learning rate schedule initialized at 5×10^{-4} and train the model for 15 epochs.

4.2 Metrics

To evaluate both the syntactic correctness and geometric fidelity of our generated outputs, we employ a comprehensive protocol tailored to distinct task categories.

Table 4: Quantitative comparison on Cross-View Translation tasks. GPT-4o is enhanced with few-shot in-context learning. Specifically, each prompt comprises five exemplars randomly chosen from the training set. These exemplars include instructions and answer. **Red** and **green** cells denote the best and second-best results, respectively.

Method	$\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$						$\mathcal{V}_{ortho} \rightarrow \{v_{iso}\}$					
	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$
GPT-4o(5shot)	46.44%	35.54%	33.53	0.80	0.19	0.34	11.33%	56.68%	11.04	0.72	0.23	0.23
Zero123	—	—	24.93	0.85	0.16	0.75	—	—	—	—	—	—
OmniSVG	—	—	20.01	0.88	0.10	0.47	—	—	—	—	—	—
DrawingsDreamer	89.77%	80.82%	50.16	0.94	0.03	0.02	73.53%	68.36%	32.03	0.88	0.06	0.02

Conditional Generation. For translation and completion tasks (Iso-to-Ortho, Ortho-to-Iso, and Partial View Completion), we evaluate performance across three levels. Syntactically, we compute Command Accuracy (Acc_{cmd}) and Parameter Accuracy (Acc_{params}) to assess the exact match rate of structural tokens and the precision of numerical coordinates, respectively. Visually, we report PSNR, SSIM, and LPIPS on rasterized 2D multi-view SVGs. Geometrically, we evaluate cross-view geometric fidelity by uniformly sampling point clouds from the geometric paths of the generated SVGs across different views and computing the Chamfer Distance (CD) between them.

Unconditional Generation. For this task, we measure the diversity and distributional alignment of the generated drawings against the dataset. Following standard shape generation protocols [57], we report Jensen-Shannon Divergence (JSD), Coverage (COV), and Maximum Mean Discrepancy (MMD).

4.3 Performance Comparison with Existing Methods

Baselines. As direct baselines for this novel task are absent, we benchmark **DrawingsDreamer** against three representative paradigms: **GPT-4o (5-shot)** representing LLMs via in-context learning, **OmniSVG** [63] for recent SVG generation, and **Zero123** [31] serving as a Iso-to-Ortho synthesis method.

Quantitative Results. The quantitative results demonstrate that our proposed unified model consistently achieves state-of-the-art performance across all metrics. **In terms of Cross-View Translation** (Table 4), we evaluate the model’s capabilities through $\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$ and $\mathcal{V}_{ortho} \rightarrow \{v_{iso}\}$ tasks.

DrawingsDreamer significantly outperforms strong baselines, including GPT-4o, Zero123 [31], and OmniSVG [63]. The performance gap, particularly with OmniSVG, can largely be attributed to its reliance on vision encoders pre-trained on natural images, which struggle to capture the rigorous spatial alignments and precise topological dependencies required for engineering drawings. Notably, our model maintains robust syntactic fidelity, achieving superior command and parameter accuracies across both translation directions.

Table 3: Quantitative comparison on Unconditional Generation ($\emptyset \rightarrow \mathcal{V}$). JSD and MMD are multiplied by 10^2 . **Red** and **green** cells denote the best and second-best results, respectively.

Method	JSD $\downarrow$	COV $\uparrow$	MMD $\downarrow$
GPT-4o (5-shot)	1.17	33.12%	8.35
DrawingsDreamer	0.61	66.24%	5.64

For Partial View Completion (Table 6), which evaluates local geometric restoration, **DrawingsDreamer** excels in maintaining geometric fidelity and structural consistency within complex multi-view contexts. **Finally, regarding Unconditional Generation** (Table 3), our model achieves competitive JSD and MMD scores, demonstrating high generative diversity and distributional alignment. This confirms that our autoregressive paradigm effectively captures the underlying distribution of structured CAD blueprints, enabling the synthesis of complex engineering geometries from scratch.

Qualitative Results. Figure 4 illustrates the $\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$ translation, where **DrawingsDreamer** accurately reconstructs 3D geometries from isometric perspectives into aligned orthographic projections. Conversely, Figure 5 shows the inverse task ($\mathcal{V}_{ortho} \rightarrow \{v_{iso}\}$), demonstrating the model’s ability to aggregate disjoint coordinates into a consistent isometric view. Furthermore, Figure 6

Table 5: Ablation Study and Quantitative Results. The evaluation is grouped by task: Cross-View Translation (top) and Partial View Completion alongside Unconditional Generation (bottom). LPIPS, CD, JSD, and MMD are multiplied by 10^2 . Red and green cells denote the best and second-best results, respectively.

Method	$\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$						$\{v_{ortho}\} \rightarrow \mathcal{V}_{iso}$					
	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$
w/o Path Mask	86.35%	75.64%	48.99	0.91	3.53	3.17	73.18%	62.89%	31.77	0.84	6.51	2.85
Prefix Format	88.11%	80.58%	47.12	0.87	4.01	2.83	67.11%	67.28%	30.60	0.86	6.67	2.74
Random Schedule	87.49%	73.42%	40.91	0.92	4.52	3.03	70.91%	68.23%	31.64	0.83	6.43	2.59
Isolated: $\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$	88.71%	79.22%	47.42	0.94	3.44	2.36	—	—	—	—	—	—
Isolated: $\{v_{ortho}\} \rightarrow \mathcal{V}_{iso}$	—	—	—	—	—	—	75.58%	66.85%	33.68	0.89	5.58	2.45
Ours	89.77%	80.82%	50.16	0.94	3.30	2.41	73.53%	68.36%	32.03	0.88	5.51	2.38

Method	$\{v_{iso}, v_{front}\} \rightarrow \{v_{top}, v_{right}\}$						$\emptyset \rightarrow \mathcal{V}$		
	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$	JSD $\downarrow$	COV $\uparrow$	MMD $\downarrow$
w/o Path Mask	89.24%	79.91%	52.19	0.95	1.87	1.56	0.63	60.77%	5.96
Prefix Format	88.38%	80.07%	57.86	0.90	1.99	1.53	0.67	62.45%	5.70
Random Schedule	92.18%	86.39%	57.19	0.96	2.02	1.48	0.88	61.21%	5.85
Isolated: Partial	85.02%	83.18%	55.52	0.96	2.31	1.73	—	—	—
Isolated: Uncond.	—	—	—	—	—	—	0.71	60.38%	5.79
Ours	92.82%	82.52%	58.61	0.97	1.83	1.41	0.61	66.24%	5.64

highlights our model’s localized spatial reasoning on partial view completion. Even with complex internal contours, the generated views maintain strict geometric and structural alignment with the provided multi-view context.

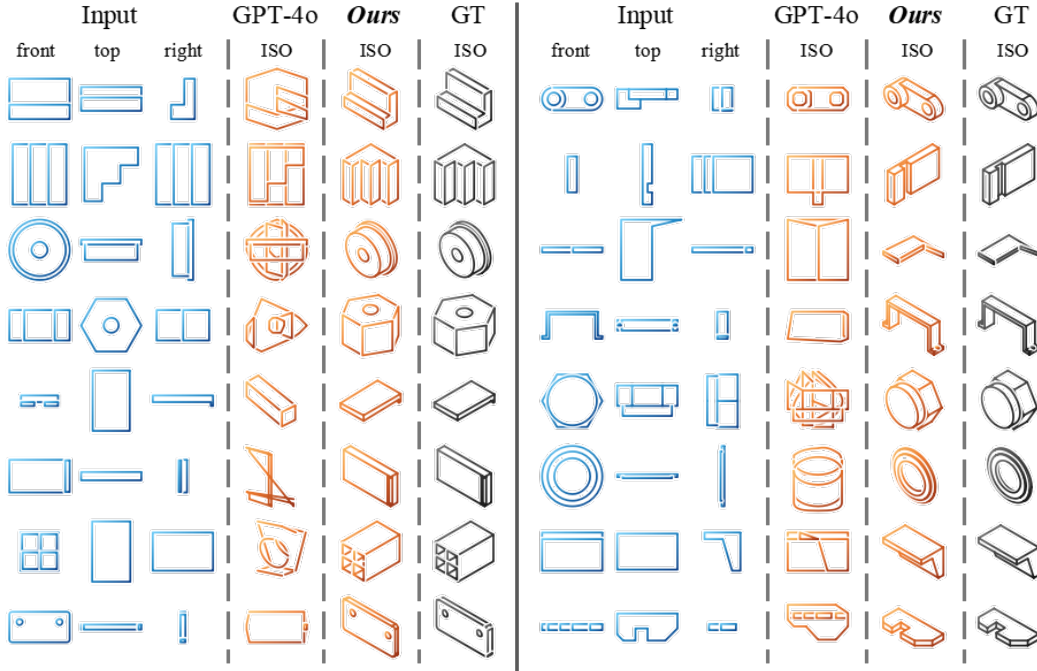


Figure 5: Qualitative results on Orthographic to Isometric translation ($\mathcal{V}_{ortho} \rightarrow \{v_{iso}\}$).

4.4 Ablation Studies

To rigorously evaluate our proposed components, we conduct comprehensive ablation studies as summarized in Table 5.

Unified Training vs. Isolated Optimization. Compared to training specialized models for isolated tasks (e.g., $\{v_{iso}\} \rightarrow \mathcal{V}_{ortho}$), our unified multitask paradigm consistently achieves superior perfor-

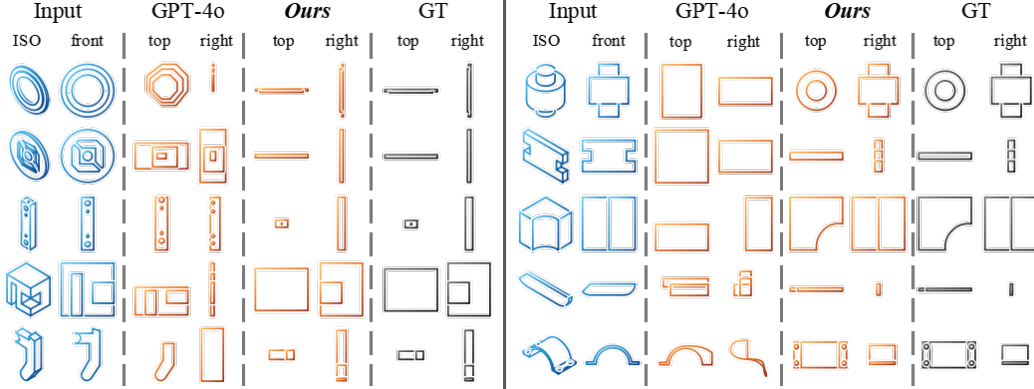


Figure 6: Qualitative results on Partial View Completion ($\{v_{\text{iso}}, v_{\text{front}}\} \rightarrow \{v_{\text{top}}, v_{\text{right}}\}$). Both the left and right panels display separate generation cases.

Table 6: Quantitative evaluation on Partial View Completion ($\{v_{\text{iso}}, v_{\text{front}}\} \rightarrow \{v_{\text{top}}, v_{\text{right}}\}$). Red and green cells denote the best and second-best results

Method	$Acc_{cmd} \uparrow$	$Acc_{params} \uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$
GPT-4o (5-shot)	65.48%	30.13%	16.52	0.83	0.16	0.30
DrawingsDreamer	92.82%	82.52%	58.61	0.97	0.02	0.01

mance. This confirms that joint optimization effectively transfers knowledge across tasks to enhance overall multi-view spatial reasoning.

Task-Aware Curriculum Schedule. Replacing our progressive three-stage curriculum with random task sampling leads to sub-optimal convergence and degraded Chamfer Distance. This highlights the necessity of a structured transition, anchoring local syntax before scaling to global spatial translation, to stabilize the generative process.

Path Level Semantic Masking. Removing this microscopic structural task during training degrades geometric fidelity and command accuracy in the generated multiview sequences. Forcing the model to explicitly learn basic geometric structures serves as a crucial stepping stone for coherent macroscopic generation.

Postfix Tokenization vs. Prefix Formats. Replacing our hierarchical postfix tokenization with standard prefix representations (e.g., M, L, C sequencing) drops parameter accuracy. Regressing coordinates before assigning semantic boundaries aligns better with the innate sequence modeling capabilities of causal LLMs in engineering drawing scenarios.

5 Conclusion

In this paper, we introduced **DrawingsDreamer**, a unified generative framework that frames multi-view engineering drawing synthesis as a pure sequence modeling task. By leveraging a streamlined representation with hierarchical postfix tokenization and a progressive task-aware curriculum, our approach effectively captures rigorous cross-view spatial alignments and complex geometric dependencies. Extensive evaluations demonstrate that **DrawingsDreamer** sets a new state of the art in generating geometrically precise parametric engineering drawings. We anticipate that expanding its geometric vocabulary to support more complex parametric constraints will further advance the integration of general-purpose sequence modeling into professional AI-assisted industrial design workflows.

References

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] K. Alrashedy, P. Tambwekar, Z. H. Zaidi, M. Langwasser, W. Xu, and M. C. Gombolay. Generating CAD code with vision-language models for 3d designs. In *ICLR*, 2025.
- [3] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *ICML*, 2009.
- [4] J. D. Camba, P. Company, and F. Naya. Sketch-based modeling in mechanical engineering design: Current status and opportunities. *Computer-Aided Design*, 2022.
- [5] A. Carlier, M. Danelljan, A. Alahi, and R. Timofte. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems*, 2020.
- [6] C. Chen, J. Wei, T. Chen, C. Zhang, X. Yang, S. Zhang, B. Yang, C.-S. Foo, G. Lin, Q. Huang, et al. Cadcrafter: Generating computer-aided design models from unconstrained images. In *CVPR*, 2025.
- [7] L. Clouâtre and M. Demers. Figr: Few-shot image generation with reptile. *arXiv preprint arXiv:1901.02199*, 2019.
- [8] B. C. Das, M. H. Amini, and Y. Wu. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 2025.
- [9] Y. Dong, J. Ding, X. Jiang, G. Li, Z. Li, and Z. Jin. Codescore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [10] R. Furferi, L. Governi, M. Palai, Y. Volpe, et al. From 2d orthographic views to 3d pseudo-wireframe: An automatic procedure. *International Journal of Computer Applications*, 2010.
- [11] J.-H. Gong, G.-F. Zhang, H. Zhang, and J.-G. Sun. Reconstruction of 3d curvilinear wire-frame from three orthographic views. *Computers & Graphics*, 2006.
- [12] X. Gu, M. Chen, Y. Lin, Y. Hu, H. Zhang, C. Wan, Z. Wei, Y. Xu, and J. Wang. On the effectiveness of large language models in domain-specific code generation. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [13] D. Ha and D. Eck. A neural representation of sketch drawings. In *ICLR*, 2018.
- [14] A. B. Harish and A. R. Prasad. Photo2cad: Automated 3d solid reconstruction from 2d drawings using opencv. *arXiv preprint arXiv:2101.04248*, 2021.
- [15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 2022.
- [16] T. Hu, R. Yi, B. Qian, J. Zhang, P. L. Rosin, and Y.-K. Lai. Supersvg: Superpixel-based scalable vector graphics synthesis. In *CVPR*, 2024.
- [17] B. Jiang, X. Chen, W. Liu, J. Yu, G. Yu, and T. Chen. Motiongpt: Human motion as a foreign language. *NeurIPS*, 2023.
- [18] M. S. Khan, E. Dupont, S. A. Ali, K. Cherenkova, A. Kacem, and D. Aouada. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. In *CVPR*, 2024.
- [19] M. S. Khan, M. Usama, R. A. Potamias, D. Stricker, M. Z. Afzal, J. Deng, and I. Elezi. Dreamcad: Scaling multi-modal cad generation using differentiable parametric surfaces. *arXiv preprint arXiv:2603.05607*, 2026.
- [20] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, Y. Jernite, M. Mitchell, C. M. Ferrandis, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries. The stack: 3 TB of permissively licensed source code. *Trans. Mach. Learn. Res.*, 2023.
- [21] S. Koch, A. Matveev, Z. Jiang, F. Williams, A. Artemov, E. Burnaev, M. Alexa, D. Zorin, and D. Panozzo. Abc: A big cad model dataset for geometric deep learning. In *CVPR*, 2019.
- [22] M.-H. Kuo. Reconstruction of quadric surface solids from three-view engineering drawings. *Computer-Aided Design*, 1998.
- [23] M. Lee, D. Zhang, C. Jambon, and Y. M. Kim. Brepdiff: Single-stage b-rep diffusion model. In *SIGGRAPH*, 2025.

- [24] J. Li, Y. Fu, and F. Chen. Dtgbreppen: A novel b-rep generative model through decoupling topology and geometry. In *CVPR*, 2025.
- [25] J. Li, K. Pan, Z. Ge, M. Gao, W. Ji, W. Zhang, T. Chua, S. Tang, H. Zhang, and Y. Zhuang. Fine-tuning multimodal llms to follow zero-shot demonstrative instructions. In *ICLR*, 2024.
- [26] P. Li, W. Zhang, J. Chen, and D. Yan. Stitch-a-shape: Bottom-up learning for b-rep generation. In *SIGGRAPH*, 2025.
- [27] P. Li, W. Zhang, J. Guo, J. Chen, and D.-M. Yan. Revisiting cad model generation by learning raster sketch. In *AAAI*, 2025.
- [28] P. Li, W. Zhang, W. Quan, B. Zhang, P. Wonka, and D. Yan. Brepopt: Autoregressive b-rep generation with voronoi half-patch. *TOG*, 2025.
- [29] T.-M. Li, M. Lukáč, M. Gharbi, and J. Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *TOG*, 2020.
- [30] Y. Li, C. Lin, Y. Liu, X. Long, C. Zhang, N. Wang, X. Li, W. Wang, and X. Guo. Caddreamer: Cad object generation from single-view images. In *CVPR*, 2025.
- [31] R. Liu, R. Wu, B. Van Hoorick, P. Tokmakov, S. Zakharov, and C. Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9298–9309, 2023.
- [32] S. Liu, W. Chen, and A. Wu. Hidigen: Hierarchical diffusion for b-rep generation with explicit topological constraints. *arXiv preprint arXiv:2604.02847*, 2026.
- [33] S.-X. Liu, S.-M. Hu, Y.-J. Chen, and J.-G. Sun. Reconstruction of curved solids from engineering drawings. *Computer-Aided Design*, 2001.
- [34] R. G. Lopes, D. Ha, D. Eck, and J. Shlens. A learned representation for scalable vector graphics. In *ICCV*, 2019.
- [35] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [36] X. Ma, Y. Zhou, X. Xu, B. Sun, V. Filev, N. Orlov, Y. Fu, and H. Shi. Towards layer-wise image vectorization. In *CVPR*, 2022.
- [37] A. Meta. Introducing meta llama 3: The most capable openly available llm to date. *Meta AI*, 2024.
- [38] I. Puhachov, C. Martens, P. G. Kry, and M. Bessmeltsev. Reconstruction of machine-made shapes from bitmap sketches. *TOG*, 2023.
- [39] F. Qin, S. Lu, J. Hou, C. Wang, M. Fang, and L. Liu. Drawing2cad: Sequence-to-sequence learning for cad generation from vector drawings. In *ACM MM*, 2025.
- [40] P. Reddy, M. Gharbi, M. Lukac, and N. J. Mitra. Im2vec: Synthesizing vector graphics without vector supervision. In *CVPR*, 2021.
- [41] J. A. Rodriguez, A. Puri, S. Agarwal, I. H. Laradji, P. Rodriguez, S. Rajeswar, D. Vazquez, C. Pal, and M. Pedersoli. Starvector: Generating scalable vector graphics code from images and text. In *CVPR*, 2025.
- [42] D. Rukhovich, E. Dupont, D. Mallis, K. Cherenkova, A. Kacem, and D. Aouada. Cad-recode: Reverse engineering cad code from point clouds. In *ICCV*, 2025.
- [43] P. Shojaei, K. Meidani, S. Gupta, A. B. Farimani, and C. K. Reddy. LLM-SR: scientific equation discovery via programming with large language models. In *ICLR*, 2025.
- [44] Y. Song, X. Shao, K. Chen, W. Zhang, Z. Jing, and M. Li. Clipvg: Text-guided image manipulation using differentiable vector graphics. In *AAAI*, 2023.
- [45] H. Su, X. Liu, J. Niu, J. Cui, J. Wan, X. Wu, and N. Wang. Marvel: Raster gray-level manga vectorization via primitive-wise deep reinforcement learning. *TCSVT*, 2023.
- [46] Z. Tang, C. Wu, Z. Zhang, M. Ni, S. Yin, Y. Liu, Z. Yang, L. Wang, Z. Liu, J. Li, and N. Duan. Strokenuwa - tokenizing strokes for vector graphic synthesis. In *ICML*, 2024.
- [47] Y. Tian and D. Ha. Modern evolution strategies for creativity: Fitting concrete images and abstract concepts. In *EvoMUSART*, 2022.

- [48] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [49] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *NeurIPS*, 2017.
- [50] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [51] S. Wang, C. Chen, X. Le, Q. Xu, L. Xu, Y. Zhang, and J. Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. In *AAAI*, 2025.
- [52] W. Wang and G. G. Grinstein. A survey of 3d solid reconstruction from 2d projection line drawings. In *Computer Graphics Forum*. Wiley Online Library, 1993.
- [53] X. Wang, J. Zheng, Y. Hu, H. Zhu, Q. Yu, and Z. Zhou. From 2d cad drawings to 3d parametric models: A vision-language approach. In *AAAI*, 2025.
- [54] Y. Wang and Z. Lian. Deepvecfont: synthesizing high-quality vector fonts via dual-modality learning. *TOG*, 2021.
- [55] K. D. Willis, Y. Pu, J. Luo, H. Chu, T. Du, J. G. Lambourne, A. Solar-Lezama, and W. Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *TOG*, 2021.
- [56] R. Wu, W. Su, and J. Liao. Chat2svg: Vector graphics generation with large language models and image diffusion models. In *CVPR*, 2025.
- [57] R. Wu, C. Xiao, and C. Zheng. Deepcad: A deep generative network for computer-aided design models. In *ICCV*, 2021.
- [58] S. Wu, A. H. Khasahmadi, M. Katz, P. K. Jayaraman, Y. Pu, K. Willis, and B. Liu. Cadvlm: Bridging language and vision in the generation of parametric cad sketches. In *ECCV*. Springer, 2024.
- [59] X. Xing, J. Hu, G. Liang, J. Zhang, D. Xu, and Q. Yu. Empowering llms to understand and generate complex vector graphics. In *CVPR*, 2025.
- [60] X. Xu, P. Jayaraman, J. Lambourne, Y. Liu, D. Malpure, and P. Meltzer. Autobrep: Autoregressive b-rep generation with unified topology and geometry. In *SIGGRAPH*, 2025.
- [61] X. Xu, J. Lambourne, P. Jayaraman, Z. Wang, K. Willis, and Y. Furukawa. Brepjen: A b-rep generative diffusion model with structured latent geometry. *TOG*, 2024.
- [62] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [63] Y. Yang, W. Cheng, S. Chen, X. Zeng, F. Yin, J. Zhang, L. Wang, G. Yu, X. Ma, and Y.-G. Jiang. Omnisvg: A unified scalable vector graphics generation model. In *NeurIPS*. Curran Associates, Inc., 2025.
- [64] M. Yavartanoo, S. Hong, R. Neshatavar, and K. M. Lee. Text2cad: Text to 3d cad generation via technical drawings. *arXiv preprint arXiv:2411.06206*, 2024.
- [65] C. Zhang, R. Piquié, A. Polette, G. Carasi, H. De Charnace, and J.-P. Pernot. Automatic 3d cad models reconstruction from 2d orthographic drawings. *Computers & Graphics*, 2023.
- [66] R. Zhang, J. Han, C. Liu, A. Zhou, P. Lu, Y. Qiao, H. Li, and P. Gao. Llama-adapter: Efficient fine-tuning of large language models with zero-initialized attention. In *ICLR*, 2024.
- [67] Y. Zhang, F. Feng, J. Zhang, K. Bao, Q. Wang, and X. He. Collm: Integrating collaborative embeddings into large language models for recommendation. *TKDE*, 2025.

Appendix

Due to space limitations in the main paper, we provide additional results and discussions in this appendix, organized as follows:

- Sec. A Data Processing Details
- Sec. B Prompting Templates and Task Configurations
- Sec. C The Impact of Model Capacity
- Sec. D Evaluation Metric Details
- Sec. E Additional Qualitative Results
- Sec. F Limitations and Future Work

A Data Processing Details

Given a CAD program $\mathcal{J}$, we first reconstruct its corresponding three-dimensional solid $\mathcal{B}$. We then generate a set of fixed-view two-dimensional drawings from $\mathcal{B}$. The view set consists of three orthographic views, namely Front, Right, and Top, together with one isometric view denoted as Iso. Samples with missing or empty views are discarded.

To obtain geometrically stable multi-view inputs, all views are normalized into a common $L \times L$ canvas, where $L = 1000$. The three orthographic views share a single scale factor, computed from their maximum spatial extent, so that their relative sizes remain consistent across views. The isometric view is normalized independently because its projected extent differs from the orthographic projections.

Before tokenization, we canonicalize the SVG paths in each view. Whenever closed contours can be recovered, we group path segments into loops and sort the loops according to their spatial positions. For views with open or more complex path structures, we use a deterministic traversal order while preserving all visible path segments. This reduces the ambiguity introduced by arbitrary SVG export order.

Each canonicalized path is then serialized into tokens. We retain three SVG drawing commands: move, line, and cubic Bézier curve, corresponding to M , L , and C , respectively. For a continuous coordinate $x \in [0, L]$, its absolute quantized value is computed as

$$q(x) = \text{clip} \left(\left\lfloor \frac{Qx}{L} \right\rfloor, 0, Q \right), \quad (3)$$

where $Q = 256$ is the coordinate quantization resolution. The four view token sequences are concatenated in the order Front, Right, Top, and Iso, with a view-specific ending token inserted after each view. The resulting sequence $\mathcal{S}$ is stored together with the model identifier and quantization resolution as one training instance.

B Prompting Templates and Task Configurations

As described in Section 4.3, our evaluation of GPT-4o employs a 5-shot in-context learning setup. Figure A1 illustrates the detailed prompt structure used in our experiments, which includes the system instruction and the layout of the five randomly sampled input-output exemplars prepended to the target query.

As introduced in Section 3.1, **DrawingsDreamer** unifies diverse generation tasks into a single autoregressive framework via task-specific instruction prompts ($\mathcal{I}$). Figure A2 provides the exact textual instruction templates prepended to the context sequences (S_{context}) for each multitask configuration.

You are a CAD/SVG expert. All coordinates are quantized into 128 bins (0-127).
Data Format: Views are concatenated strings ending with specific tags (<end_front>, etc.).
IMPORTANT: Your response must be a single concatenated string of the target views, maintaining the strict order: Front -> Right -> Top -> ISO (only for the requested views).

Instruction
Synthesize the 3D Isometric (ISO) view sequence.

Examples

Example 1:
[INPUT]: <front view> 27 27 <end_move> 49 27 <end_line> 78 27 <end_line> 100 27 <end_line> 100 42 <end_line> 78 42 <end_line> 78 27 <end_line> 27 27 <end_move> 27 42 <end_line> 49 42 <end_line> 49 27 <end_line> 27 42 <end_move> 27 64 <end_line> 27 84 43 100 64 100 <end_curve> 84 100 100 84 100 64 <end_curve> 100 49 <end_line> 93 49 <end_line> 93 42 <end_line> 49 42 <end_move> 78 42 <end_line> <end_path> <end_front> <right view> 9 27 <end_move> 89 27 <end_line> 89 42 <end_line> 9 42 <end_line> 9 27 <end_line> <end_path> 89 27 <end_move> 118 27 <end_line> 118 42 <end_line> 118 49 <end_line> 103 49 <end_line> 103 42 <end_line> 89 42 <end_line> 89 27 <end_line> <end_path> 103 49 <end_move> 118 49 <end_line> 118 64 <end_line> 103 64 <end_line> 103 49 <end_line> <end_path> 103 64 <end_move> 118 64 <end_line> 118 100 <end_line> 103 100 <end_line> 103 64 <end_line> <end_path> <end_right> <top view> 27 9 <end_move> 93 9 <end_line> 100 9 <end_line> 100 24 <end_line> 93 24 <end_line> 27 9 <end_move> 27 118 <end_line> 49 118 <end_line> 49 82 <end_line> 49 74 55 67 64 67 <end_curve> 72 67 78 74 78 82 <end_curve> 78 118 <end_line> 100 118 <end_line> 100 38 <end_line> 93 38 <end_line> 93 9 <end_line> <end_path> <end_top>
[TARGET RESPONSE]: 74 27 <end_move> 114 49 <end_line> 96 60 <end_line> 100 62 <end_line> 100 72 <end_line> 53 100 <end_line> 53 90 <end_line> 100 62 <end_line> 74 27 <end_move> 9 65 <end_line> 22 72 <end_line> 44 60 <end_line> 47 57 53 57 57 58 <end_curve> 62 59 65 62 65 65 <end_curve> 65 66 64 68 61 70 <end_curve> 57 67 49 67 44 70 <end_curve> 44 60 <end_line> 114 49 <end_move> 114 60 <end_line> 114 65 <end_line> 118 67 <end_line> 109 72 <end_line> 105 70 <end_line> 105 65 <end_line> 100 67 <end_line> 9 65 <end_move> 9 75 <end_line> 22 82 <end_line> 44 70 <end_line> 114 65 <end_move> 109 82 <end_line> 105 70 <end_line> 22 72 <end_move> 22 82 <end_line> 118 77 <end_move> 118 67 <end_line> 118 77 <end_move> 109 82 <end_line> 109 72 <end_line> 40 82 <end_move> 61 70 <end_line> 40 82 <end_move> 53 90 <end_line> 40 82 <end_move> 40 92 <end_line> 53 100 <end_line> 77 86 <end_move> 86 96 96 100 103 96 <end_curve> 107 94 109 89 109 82 <end_curve> 112 91 <end_move> 116 89 118 84 118 77 <end_curve> 112 91 <end_move> 103 96 <end_line> <end_path> <end_iso>

Example 2:
[INPUT]: <front view> 9 59 <end_move> 118 59 <end_line> 118 68 <end_line> 9 68 <end_line> 9 59 <end_line> <end_path> <end_front> <right view> 51 59 <end_move> 76 59 <end_line> 76 68 <end_line> 51 68 <end_line> 51 59 <end_line> <end_path> <end_right> <top view> 9 51 <end_move> 118 51 <end_line> 118 76 <end_line> 83 64 44 64 9 76 <end_curve> 9 51 <end_line> <end_path> <end_top>
[TARGET RESPONSE]: 30 28 <end_move> 118 79 <end_line> 97 91 <end_line> 79 69 48 51 9 40 <end_curve> 30 28 <end_line> <end_path> 9 40 <end_move> 48 51 79 69 97 91 <end_curve> 97 99 <end_line> 79 77 48 59 9 48 <end_curve> 9 40 <end_line> <end_path> 118 79 <end_move> 118 87 <end_line> 97 99 <end_line> 97 91 <end_line> 118 79 <end_line> <end_path> <end_iso>

Example 3:
[INPUT]: <front view> 9 31 <end_move> 118 31 <end_line> 118 96 <end_line> 9 96 <end_line> 9 31 <end_line> <end_path> <end_front> <right view> 35 31 <end_move> 92 31 <end_line> 92 96 <end_line> 35 96 <end_line> 35 31 <end_line> <end_path> <end_right> <top view> 9 35 <end_move> 118 35 <end_line> 118 92 <end_line> 9 92 <end_line> 9 35 <end_line> <end_path> <end_top>
[TARGET RESPONSE]: 47 9 <end_move> 116 49 <end_line> 80 70 <end_line> 11 30 <end_line> 47 9 <end_line> <end_path> 11 30 <end_move> 80 70 <end_line> 80 118 <end_line> 11 78 <end_line> 11 30 <end_line> <end_path> 116 49 <end_move> 116 97 <end_line> 80 118 <end_line> 80 70 <end_line> 116 49 <end_line> <end_path> <end_iso>

Example 4:
[INPUT]: <front view> 9 9 <end_move> 118 9 <end_line> 118 118 <end_line> 9 118 <end_line> 9 9 <end_line> <end_path> <end_front> <right view> 9 9 <end_move> 118 9 <end_line> 118 118 <end_line> 9 118 <end_line> 9 9 <end_line> <end_path> <end_right> <top view> 9 9 <end_move> 118 9 <end_line> 118 118 <end_line> 9 118 <end_line> 9 9 <end_line> <end_path> <end_top>
[TARGET RESPONSE]: 64 9 <end_move> 111 36 <end_line> 64 64 <end_line> 16 36 <end_line> 64 9 <end_line> <end_path> 16 36 <end_move> 64 64 <end_line> 64 118 <end_line> 16 91 <end_line> 16 36 <end_line> <end_path> 111 36 <end_move> 111 91 <end_line> 64 118 <end_line> 64 64 <end_line> 111 36 <end_line> <end_path> <end_iso>

Example 5:
[INPUT]: <front view> 9 36 <end_move> 118 36 <end_line> 118 91 <end_line> 9 91 <end_line> 9 36 <end_line> <end_path> 27 54 <end_move> 45 54 <end_line> 45 73 <end_line> 27 73 <end_line> 27 54 <end_line> <end_path> 64 54 <end_move> 74 73 <end_line> 64 73 <end_line> 53 73 <end_line> 64 54 <end_line> <end_path> 91 54 <end_move> 96 54 100 58 100 64 <end_curve> 100 69 96 73 91 73 <end_curve> 86 73 82 69 82 64 <end_curve> 82 58 86 54 91 54 <end_curve> <end_path> <end_front> <right view> 54 36 <end_move> 73 36 <end_line> 73 91 <end_line> 54 91 <end_line> 54 36 <end_line> <end_path> <end_right> <top view> 9 54 <end_move> 118 54 <end_line> 118 73 <end_line> 9 73 <end_line> 9 54 <end_line> <end_path> <end_top>
[TARGET RESPONSE]: 27 9 <end_move> 13 17 <end_line> 13 68 <end_line> 100 118 <end_line> 100 68 <end_line> 13 17 <end_line> 114 59 <end_move> 27 9 <end_line> 114 59 <end_move> 100 68 <end_line> 114 110 <end_move> 114 59 <end_line> 114 110 <end_move> 100 118 <end_line> <end_path> 27 43 <end_move> 42 51 <end_line> 27 59 <end_line> 27 43 <end_line> 42 68 <end_move> 42 51 <end_line> 42 68 <end_move> 27 59 <end_line> <end_path> 56 59 <end_move> 48 71 <end_line> 58 65 <end_line> 48 71 <end_move> 56 76 <end_line> 65 81 <end_line> 56 59 <end_line> <end_path> 73 71 <end_move> 75 70 78 71 81 74 <end_curve> 83 77 85 81 85 84 <end_curve> 85 87 84 88 83 89 <end_curve> 80 91 76 88 73 83 <end_curve> 70 78 70 73 73 71 <end_curve> <end_path> <end_iso>

Current Test Task
[INPUT]: <front view> 36 9 <end_move> 91 9 <end_line> 91 118 <end_line> 36 118 <end_line> 36 9 <end_line> <end_path> <end_front> <right view> 36 9 <end_move> 91 9 <end_line> 91 118 <end_line> 36 118 <end_line> 36 9 <end_line> <end_path> <end_right> <top view> 64 36 <end_move> 79 36 91 48 91 64 <end_curve> 91 79 79 91 64 91 <end_curve> 48 91 36 79 36 64 <end_curve> 36 48 48 36 64 36 <end_curve> <end_path> 64 46 <end_move> 73 46 81 54 81 64 <end_curve> 81 73 73 81 64 81 <end_curve> 54 81 46 73 46 64 <end_curve> 46 54 54 46 64 46 <end_curve> <end_path> <end_top>
[TARGET RESPONSE]:

Figure A1: The 5-shot in-context learning prompt template used for the GPT-4o baseline evaluation.

Unconditional Generation

Below is an instruction to restore missing CAD views.

Instruction: Generate standard 4-view SVG sequence.

Target Response:

```
59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path> <end_front>
38 9 <end_move> 89 9 <end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line> <end_path> <end_right>
69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line> <end_path> <end_top> 82 9 <end_move> 82
97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9 <end_curve> <end_path> <end_iso>
```

Front, Top and Right Views Generation from ISO View

Below is an instruction to restore missing CAD views.

Instruction: Given ISO view, generate Front, Right, and Top views.

Input Context:

```
<iso_view> 82 9 <end_move> 82 97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9 <end_curve>
<end_path> <end_iso>
```

Target Response:

```
59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path> <end_front>
38 9 <end_move> 89 9 <end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line> <end_path> <end_right>
69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line> <end_path> <end_top>
```

ISO View Generation from Front, Top and Right Views

Below is an instruction to restore missing CAD views.

Instruction: Given Front, Right, and Top views, generate ISO view.

Input Context:

```
<front_view> 59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path>
<end_front> <right_view> 38 9 <end_move> 89 9 <end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line>
<end_path> <end_right> <top_view> 69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line>
<end_path> <end_top>
```

Target Response:

```
82 9 <end_move> 82 97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9 <end_curve> <end_path>
<end_iso>
```

Front, Top Views Generation from ISO and Right Views

Below is an instruction to restore missing CAD views.

Instruction: Given Right and ISO views, generate Front and Top views.

Input Context:

```
<right_view> 38 9 <end_move> 89 9 <end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line> <end_path>
<end_right> <iso_view> 82 9 <end_move> 82 97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9
<end_curve> <end_path> <end_iso>
```

Target Response:

```
59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path> <end_front>
69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line> <end_path> <end_top>
```

Front and Right Views Generation from ISO and Top Views

Below is an instruction to restore missing CAD views.

Instruction: Given Top and ISO views, generate Front and Right views.

Input Context:

```
<top_view> 69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line> <end_path> <end_top>
<iso_view> 82 9 <end_move> 82 97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9 <end_curve>
<end_path> <end_iso>
```

Target Response:

```
59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path> <end_front>
38 9 <end_move> 89 9 <end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line> <end_path> <end_right>
```

Top and Right Views Generation from ISO and Front Views

Below is an instruction to restore missing CAD views.

Instruction: Given Front and ISO views, generate Top and Right views.

Input Context:

```
<front_view> 59 9 <end_move> 68 9 <end_line> 68 118 <end_line> 59 118 <end_line> 59 9 <end_line> <end_path>
<end_front> <iso_view> 82 9 <end_move> 82 97 <end_line> 45 118 <end_line> 45 30 <end_line> 49 19 63 11 82 9
<end_curve> <end_path> <end_iso>
```

Target Response:

```
69 38 <end_move> 58 53 58 74 69 89 <end_curve> 69 38 <end_line> <end_path> <end_top> 38 9 <end_move> 89 9
<end_line> 89 118 <end_line> 38 118 <end_line> 38 9 <end_line> <end_path> <end_right>
```

Figure A2: Task-specific instruction templates ($\mathcal{I}$) utilized during the unified multitask training phase.

Algorithm 1: Construction of multi-view SVG token sequences from a CAD model.

Data: CAD model description $\mathcal{J}$
Result: Tokenized multi-view instance (id, Q , $\mathcal{S}$)

```
1  $Q \leftarrow 256, \quad L \leftarrow 1000;$  // Quantization resolution and normalized canvas size
2  $\mathcal{B} \leftarrow \text{reconstruct\_solid}(\mathcal{J});$  // Recover the boundary representation from the CAD program
3  $\{\mathcal{R}_v\}_{v \in \mathcal{V}} \leftarrow \text{render\_views}(\mathcal{B});$  // Render standard 2D SVG projections
4 if  $\neg \text{valid\_views}(\{\mathcal{R}_v\})$  then
5 |   return  $\emptyset;$  // Discard models with missing or empty views
6 end
7  $\mathcal{V}_o \leftarrow \{\text{Front}, \text{Right}, \text{Top}\}, \quad \mathcal{V} \leftarrow \mathcal{V}_o \cup \{\text{Iso}\}$ 
8  $s_o \leftarrow \text{shared\_ortho\_scale}(\{\mathcal{R}_v\}_{v \in \mathcal{V}_o});$  // Use a common scale for orthographic views
9 foreach  $v \in \mathcal{V}$  do
10 |    $\mathcal{P}_v \leftarrow \text{parse\_svg\_paths}(\mathcal{R}_v);$  // Extract path commands and continuous coordinates
11 |   if  $v \in \mathcal{V}_o$  then
12 | |    $\tilde{\mathcal{P}}_v \leftarrow \text{normalize\_view}(\mathcal{P}_v, s_o, L);$  // Normalize orthographic views with the shared
13 | |   scale
14 |   else
15 | |    $\tilde{\mathcal{P}}_v \leftarrow \text{normalize\_view}(\mathcal{P}_v, \text{independent\_scale}(\mathcal{P}_v), L);$  // Normalize the isometric
16 | |   view independently
17 |   end
18 |    $\hat{\mathcal{P}}_v \leftarrow \text{order\_loops}(\tilde{\mathcal{P}}_v);$  // Recover closed loops when possible and impose a canonical
19 |   order
20 |    $\mathcal{S}_v \leftarrow \text{quantize\_and\_serialize}(\hat{\mathcal{P}}_v, Q, L);$  // Convert SVG commands into discrete
21 |   absolute-coordinate tokens
22 end
23  $\mathcal{S} \leftarrow \mathcal{S}_{\text{Front}} \parallel \langle \text{end\_front} \rangle \parallel \mathcal{S}_{\text{Right}} \parallel \langle \text{end\_right} \rangle \parallel \mathcal{S}_{\text{Top}} \parallel \langle \text{end\_top} \rangle \parallel \mathcal{S}_{\text{Iso}} \parallel \langle \text{end\_iso} \rangle$ 
24 return (id( $\mathcal{J}$ ),  $Q$ ,  $\mathcal{S}$ )
```

C The Impact of Model Capacity

To investigate the impact of fundamental Large Language Model capacity on spatial reasoning and multi-view alignment, we extend our framework by scaling **DrawingsDreamer** up to a 7B parameter architecture (e.g., LLaMA-3-7B).

Setup. While our primary experiments utilize a 3B parameter model for an optimal balance between performance and computational efficiency, generating highly structured engineering blueprints demands rigorous logical deduction and precise coordinate regression. By scaling up the backbone, we aim to explore whether increased parameter counts can further mitigate the topological hallucinations and dimensional drifts identified in our failure cases (Section F). The 7B model is fine-tuned using a consistent LoRA configuration ($r = 8, \alpha = 32$) and identical progressive task-aware curriculum scheduling. Due to memory constraints, the batch size is appropriately adjusted while maintaining the same effective learning rate trajectory.

Quantitative Improvements. Table 7 summarizes the performance comparison between the 3B and 7B variants. Empirically, scaling up the model yields consistent improvements across all metrics. Most notably, in the Partial View Completion task ($\{v_{\text{iso}}, v_{\text{front}}\} \rightarrow \{v_{\text{top}}, v_{\text{right}}\}$), the 7B model achieves a substantial leap in Parameter Accuracy (from 75.53% to 82.82%) and a corresponding drop in Chamfer Distance (from 2.38 to 1.92). This indicates that the larger semantic capacity of the 7B model translates directly into finer coordinate regression and stricter cross-view geometric fidelity.

Qualitative Observations. Beyond the quantitative metrics, the 7B model demonstrates enhanced robustness against the typical failure modes discussed in Section F. Specifically, the expanded context-processing capabilities significantly reduce counting errors for repetitive structures (e.g., arrays of circular holes), showcasing stronger local topological memory. Furthermore, the 7B variant exhibits tighter spatial constraints when inferring thin or elongated objects, reducing the coordinate drift that occasionally plagued the 3B model.

Table 7: Quantitative comparison between 3B and 7B parameter models on Cross-View Translation and Partial View Completion tasks. CD is multiplied by 10^2 . Red and green cells denote the best and second-best results, respectively.

Model	$\{v_{\text{iso}}\} \rightarrow \mathcal{V}_{\text{ortho}}$			$\{v_{\text{ortho}}\} \rightarrow \mathcal{V}_{\text{iso}}$		
	$Acc_{cmd} \uparrow$	$Acc_{param} \uparrow$	$CD \downarrow$	$Acc_{cmd} \uparrow$	$Acc_{param} \uparrow$	$CD \downarrow$
DrawingsDreamer (3B)	89.77%	80.82%	2.41	75.53%	68.36%	2.38
DrawingsDreamer (7B)	91.16%	85.41%	2.04	82.82%	73.51%	1.92

D Evaluation Metric Details

We evaluate *DrawingsDreamer* from two complementary perspectives: symbolic SVG correctness and raster-level visual fidelity. For conditional generation tasks, the model is evaluated only on the views that are required to be generated by the task instruction. For example, in the ISO-to-three-view task, the ISO view is treated as input and excluded from the target set, while the front, right, and top views are evaluated. Conversely, in the three-view-to-ISO task, only the ISO view is evaluated. For tasks conditioned on ISO and one additional orthographic view, the remaining two orthographic views are used as targets.

Symbolic accuracy. Each generated SVG sequence and its ground truth sequence are first split into individual views according to the view delimiters `<end_front>`, `<end_right>`, `<end_top>`, and `<end_iso>`. Within each target view, we parse the sequence into an ordered list of drawing operations and their numerical parameters. We report two symbolic accuracy metrics. The command accuracy measures whether the generated operation type matches the ground truth at the same sequence position:

$$Acc_{cmd} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[c_i = c_i^*], \quad (4)$$

where c_i and c_i^* denote the generated and ground-truth operation types, and N is the number of ground-truth operations. The parameter accuracy measures whether numerical parameters are reconstructed within a fixed coordinate tolerance τ :

$$Acc_{param} = \frac{1}{M} \sum_i \sum_j \mathbb{1}[|p_{ij} - p_{ij}^*| \leq \tau], \quad (5)$$

where p_{ij} and p_{ij}^* are generated and ground-truth parameters and M is the number of evaluated ground-truth parameters. Metrics are first averaged over target views for each sample and then averaged across the evaluation set. Missing target views are counted as failed predictions for the corresponding view.

Raster-level visual metrics. To evaluate visual fidelity, we render SVG sequences into raster images with a white background at a fixed resolution of 256×256 . All visual metrics are computed per target view and then averaged over the target views of each sample. We report PSNR, SSIM, LPIPS, and Chamfer Distance (CD). PSNR and SSIM measure pixel-level reconstruction quality, while LPIPS measures perceptual similarity using a learned image feature space. All rendered images are normalized to $[0, 1]$ before metric computation.

Chamfer Distance is computed on line pixels extracted from the rendered images. We convert each rendered image to grayscale and select foreground pixels with intensity below a fixed threshold. The resulting 2D point sets are normalized by the image resolution. Given the generated point set P and the ground truth point set Q , CD is computed as

$$CD(P, Q) = \frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|_2 + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|_2. \quad (6)$$

This metric complements raster similarity by directly measuring geometric alignment between predicted and ground-truth line drawings.

Unconditional generation metrics. For unconditional generation, there is no one-to-one ground truth target for each generated sample. We therefore evaluate the generated distribution against the test-set distribution using JSD, COV, and MMD. Each four-view SVG is converted into a joint 2D point cloud. Specifically, every view is sampled into a fixed-size point cloud from its line and curve primitives, locally normalized, and placed into a view-specific quadrant. The four view point clouds are then concatenated into a single joint representation.

Let G be the set of generated point clouds and R be the set of reference point clouds. We compute pairwise Chamfer distances between generated and reference samples. Minimum Matching Distance measures the average nearest-neighbor distance from generated samples to the reference set:

$$\text{MMD}(G, R) = \frac{1}{|G|} \sum_{g \in G} \min_{r \in R} \text{CD}(g, r). \quad (7)$$

Coverage measures the fraction of reference samples that are selected as the nearest neighbor of at least one generated sample:

$$\text{COV}(G, R) = \frac{|\{\arg \min_{r \in R} \text{CD}(g, r) : g \in G\}|}{|R|}. \quad (8)$$

MMD reflects sample quality, while COV reflects diversity. We also compute Jensen-Shannon Divergence by voxelizing all points from the generated and reference sets into a fixed 2D occupancy grid and comparing the resulting occupancy distributions:

$$\text{JSD}(P_G, P_R) = \frac{1}{2} \text{KL}(P_G \| M) + \frac{1}{2} \text{KL}(P_R \| M), \quad M = \frac{1}{2}(P_G + P_R). \quad (9)$$

Lower MMD and JSD indicate better distributional matching, while higher COV indicates better coverage of the test distribution.

Fair comparison with Zero123. Zero123 produces raster images rather than SVG command sequences, so symbolic accuracy is not applicable to this baseline. To ensure a fair visual comparison, we evaluate Zero123 using the same raster-level protocol as our SVG outputs. Each predicted target view and its corresponding ground truth view are converted to RGB, composited on a white background when necessary, resized to the same 256×256 resolution, and evaluated per view. PSNR, SSIM, LPIPS, and CD are then computed with the same definitions and averaged over the same target views. For CD, foreground line pixels are extracted using the same grayscale thresholding rule and the same bidirectional Chamfer formulation. Thus, although Zero123 and DrawingsDreamer use different output representations, all reported visual metrics are computed under an aligned per-view image-space protocol.

E Additional Qualitative Results

To further demonstrate the generative capacity of **DrawingsDreamer**, we provide additional qualitative examples of unconditional generation ($\emptyset \rightarrow \mathcal{V}$) in Figure A3. These results illustrate the model’s ability to sample directly from the learned prior distribution, successfully synthesizing diverse, structurally valid, and strictly aligned multi-view engineering blueprints without any contextual conditioning.

F Limitations and Future Work

While **DrawingsDreamer** demonstrates strong capabilities in generating multi-view engineering drawings, we observe certain limitations, primarily manifesting in two distinct failure modes as illustrated in Figures A4, A5.

Shape and Topological Reasoning Errors. The first category of failure involves structural hallucinations, most commonly taking the form of incorrect inference regarding discrete geometric features. For instance, the model may occasionally generate an incorrect number of repeating elements, such as missing a circular hole or hallucinating an extra symmetric slot. We attribute this to the known limitations of purely autoregressive models when handling highly repetitive structural tokens over extended context windows. Without explicit counting mechanisms, the attention mechanism can occasionally lose track of repeating sub-sequences.

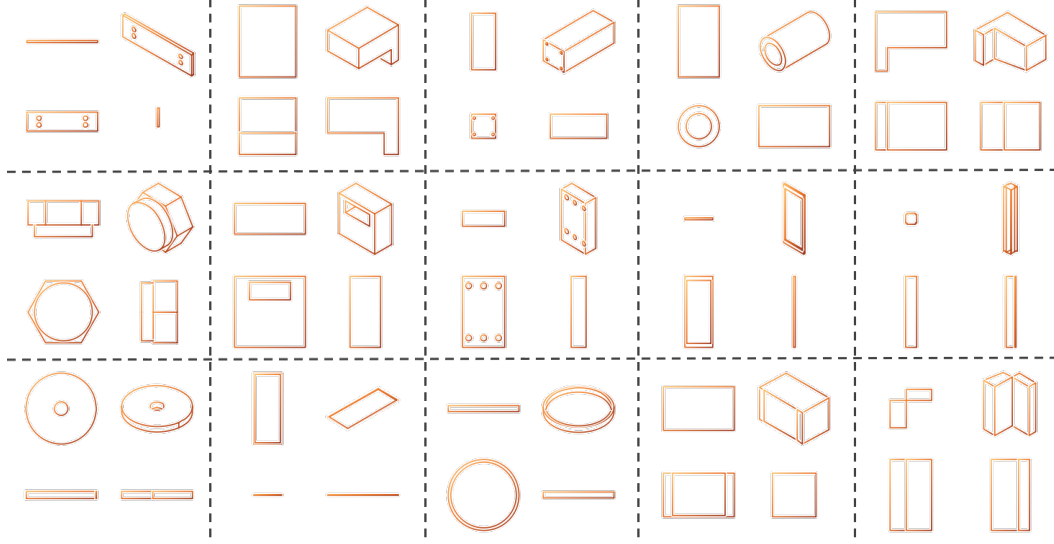


Figure A3: Qualitative results of unconditional multi-view engineering drawing generation.

Geometric and Dimensional Errors. The second category pertains to precise spatial reasoning, which is particularly pronounced when the model attempts to generate flat, thin, or highly elongated objects. In these scenarios, the model can struggle to maintain rigorous dimensional consistency across multiple views, resulting in inaccurate lengths, irregular thicknesses, distorted hole radii, or misaligned spatial coordinates. This issue stems from the inherent challenges of mapping continuous, highly skewed geometric spaces into discrete token vocabularies. For extremely thin features, a minor coordinate drift or quantization artifact in one projection can be amplified during cross-view translation, compromising the strict spatial alignment required for professional CAD blueprints.

Future Work. Addressing these limitations presents straightforward avenues for future research. While introducing explicit cross-view alignment rules could provide immediate structural constraints, a more scalable approach involves Reinforcement Learning (RL). By designing reward models that explicitly penalize cross-view geometric discrepancies, the framework can be optimized to internalize strict spatial alignment, preserving a purely data-driven sequence modeling paradigm without relying on hand-crafted heuristics.

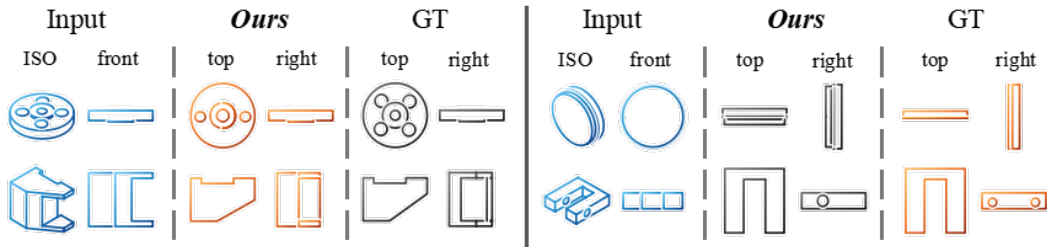


Figure A4: Failure cases during Partial View Completion.

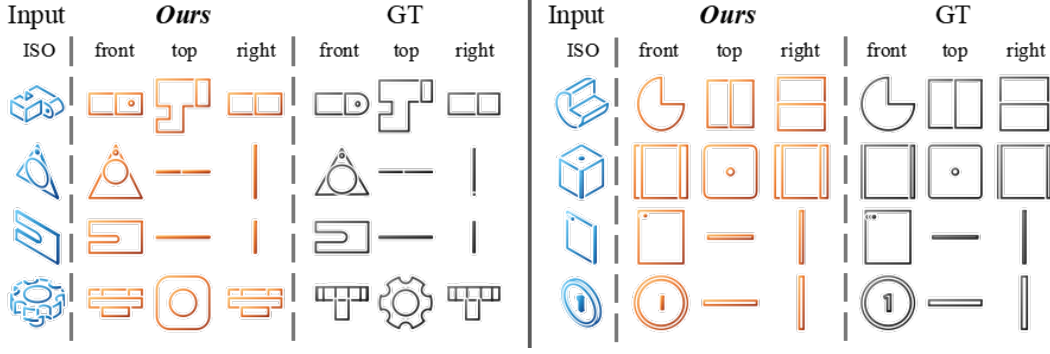


Figure A5: Failure case during Isometric to Orthographic translation.

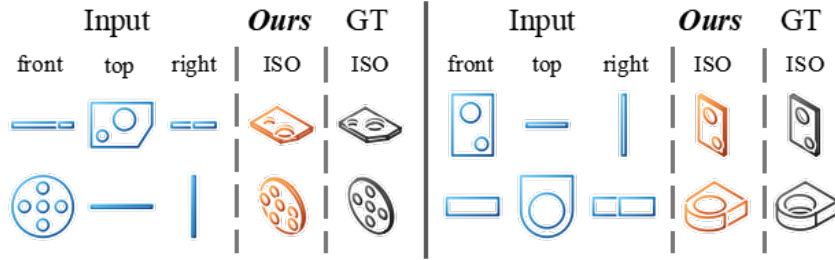


Figure A6: Failure case during Orthographic to Isometric reconstruction.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We mention our limitations in Appendix F.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.

- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We disclose all information needed for reproduction in Section 4.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.

- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: We will provide data, code, and instructions for the final paper.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: We specifies all the training and test details in Section 4.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: We report consistent performance across multiple runs and use fixed random seed settings to support the statistical significance of our results.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: We provide information about the computer resources used in Section 4.1.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have read and understood the code of ethics and have made every effort to adhere to it.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: Our work contributes to Engineering Drawings Generation. It does not impact society at large.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All existing assets, including code, data, and models, are properly cited in the paper, and their licenses and usage terms are respected in accordance with the original sources.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We will release our codebase along with detailed README files to ensure usability and reproducibility in an anonymous manner.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: This work does not use crowdsourcing or human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work does not use crowdsourcing or human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: We describe the usage of LLMs in Section 3 and details in Section 4.1.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.